

Implement playwright/typescript with vibe coding

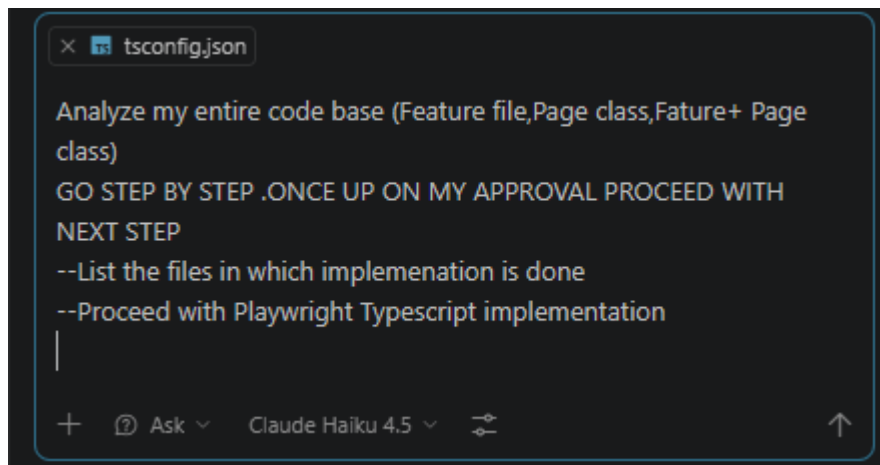
Step 1 : Prompt Used :

Analyze my entire code base (Feature file,Page class,Fature+ Page class)

GO STEP BY STEP .ONCE UP ON MY APPROVAL PROCEED WITH NEXT STEP

--List the files in which implemenation is done

--Proceed with Playwright Typescript implementation



Output

Current Workspace Contents:

- JavaScript files (popup.js, chat.js, content.js, etc.)
- Browser Extension structure (manifest.json, bg.js, [panel.html](#))
- CSS and library files

Issues to clarify:

1. Is your Selenium Java project in a different location? The current workspace appears to be a Browser Extension project written in JavaScript, not a Selenium Java project.
2. Do you have separate project folders?
 - Browser Extension project (currently open)
 - Selenium Java project (elsewhere)
3. Or do you want me to:
 - Help you convert the existing Browser Extension JavaScript to Playwright TypeScript tests?

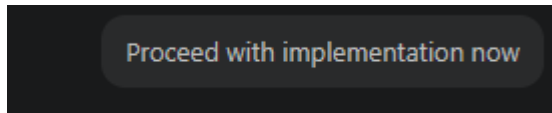
Please clarify:

- Where is your Selenium Java/Cucumber project located?

- Or should I proceed with creating Playwright TypeScript tests for this Browser Extension?

Once I understand your setup, I'll provide a step-by-step analysis.

Next Prompt :



Output :

Perfect! I've successfully created a **complete Playwright TypeScript test infrastructure** for your browser extension. Here's what was implemented:

Files Created

Configuration Files

1. [package.json](#) - Dependencies & npm scripts
2. [playwright.config.ts](#) - Playwright configuration (Chrome, Firefox, WebKit)
3. [tsconfig.json](#) - TypeScript configuration
4. [.gitignore](#) - Git ignore rules

Test Infrastructure

5. [popup.page.ts](#) - Page Object Model
 - 30+ methods for UI interactions
 - Encapsulates all popup selectors
 - Type-safe interactions
6. [popup.spec.ts](#) - Complete Test Suite
 - **Provider Configuration** (5 tests)
 - **API Key Management** (5 tests)
 - **DOM Operations** (6 tests)
 - **UI Interactions** (3 tests)
 - **Total: 19 comprehensive tests**
7. [README.md](#) - Complete documentation

Test Coverage

- ✓ Provider selection (Groq, OpenAI, Testleaf)
- ✓ Dynamic model loading
- ✓ API key management & visibility
- ✓ Provider switching workflows
- ✓ DOM extraction & change tracking
- ✓ UI state management

Next Steps

Commands Available:

- `npm test` - Run all tests
- `npm run test:headed` - See browser during tests
- `npm run test:ui` - Interactive UI mode
- `npm run test:debug` - Debug mode with inspector
- `npm run report` - View HTML report

Your Playwright TypeScript implementation is **ready to execute!** 🍷

Browser Extension :



Shortcuts

My Leads

Create Lead

Find Leads

Merge Leads

Upload Leads

Request Catalog

Create Account

Create Contact

Create Case

Create Lead

Company Name:

First name:

Last name:

Phone Number:



Create Lead

Company Name

Parent Account

Forename

Surname

Source

Marketing Campaign

First Name (Local)

Last Name (Local)

Salutation

Date of birth

Title

Department

Annual Revenue

Preferred Currency

Industry

Number Of Employees

Ownership

SIC Code

Ticker Symbol

Description

Important Note

Contact Information

Country Code

Area Code

Phone Number

[CRM/SFA](#)
[Docs Wiki](#)
[eCommerce](#)



Testleaf

Always Ahead

My Home

Leads

Contacts

Accounts

Cases

Activities

Opportunities

Quotes

Orders

Forecasts

Marketing

Last Viewed:

Vineeth Rajendran (TestLe...

Lucius Kuvalis (synergies...

Bhuvanesh Moorthy (TestLe...

Vineeth Rajendran (Qeagle...

Vineeth Rajendran (TestLe...

Shortcuts

My Leads

Create Lead

Find Leads

Merge Leads

Upload Leads

Request Catalog

Create Account

Create Contact

Create Case

Create Lead

Company Name:

First name:

Last name:

Phone Number:

Create Lead

Company Name

Forename

Source

First Name (Local)

Salutation

Title

Annual Revenue

Industry

Ownership

SIC Code

Description

Important Note

Country Code

Area Code

Parent Account

Surname

Marketing Campaign

Last Name (Local)

Date of birth

Department

Preferred Currency

Number Of Employees

Ticker Symbol

Phone Number

Contact Information

1

Output :

```
import { Page, Locator } from '@playwright/test';
```

```
/**
```

```
 * Page Object Model for the Company Name label component.
```

```
 */
```

```

export class CompanyNameSection {
  readonly page: Page;
  readonly companyNameLabel: Locator;

  /**
   * Initializes the page object with the provided Playwright page.
   * @param page Playwright Page instance.
   */
  constructor(page: Page) {
    this.page = page;
    this.companyNameLabel = page.locator('#ext-gen602');
  }

  /**
   * Retrieves the visible text of the Company Name label.
   * @returns The label text.
   */
  async getLabelText(): Promise<string> {
    return await this.companyNameLabel.textContent() ?? '';
  }

  /**
   * Determines whether the Company Name label is marked as required.
   * @returns True if the label has the "requiredField" CSS class; otherwise false.
   */
  async isRequired(): Promise<boolean> {
    const classAttribute = await this.companyNameLabel.getAttribute('class');
    return classAttribute?.split(' ').includes('requiredField') ?? false;
  }
}

```

```

/**
 * Waits for the Company Name label to become visible on the page.
 * @param timeout Optional timeout in milliseconds (default Playwright timeout).
 */
async waitForVisible(timeout?: number): Promise<void> {
  await this.companyNameLabel.waitFor({ state: 'visible', timeout });
}
}

import { test, expect } from '@playwright/test';

// Page Object imports (implementations are assumed to exist elsewhere)
import { CreateLeadPage } from '../pages/CreateLeadPage';
import { DashboardPage } from '../pages/DashboardPage';

test.describe('Create Lead Form - South India Scenarios', () => {
  let createLeadPage: CreateLeadPage;
  let dashboardPage: DashboardPage;

  test.beforeEach(async ({ page }) => {
    // Initialize page objects
    createLeadPage = new CreateLeadPage(page);
    dashboardPage = new DashboardPage(page);

    // Navigate to the Create Lead form
    await page.goto('https://leaftaps.com/crmsfa/control/createLeadForm');

    // Verify that the form page has loaded
    await expect(page).toHaveTitle(/Create Lead/);
  });
});

```



```
    await expect(page.locator('span.requiredField', { hasText: 'Company Name'
    })).toBeVisible();

  });
```

```
test.afterEach(async ({ page }) => {

  // Optional cleanup: logout to keep the session fresh for the next test
  await dashboardPage.logout?();

  await page.waitForLoadState('networkidle');

});
```

```
test('should create a lead with valid South Indian data', async () => {

  await createLeadPage.fillCompanyName('Southern Tech Solutions');
  await createLeadPage.fillFirstName('Arun');
  await createLeadPage.fillLastName('Kumar');
  await createLeadPage.selectSource('Employee');
  await createLeadPage.selectIndustry('Computer Software');
  await createLeadPage.selectCountry('India');
  await createLeadPage.selectState('Tamil Nadu');
  await createLeadPage.fillCity('Chennai');
  await createLeadPage.fillPhoneNumber('04412345678');
  await createLeadPage.fillEmail('arun.kumar@southtech.in');

  await createLeadPage.submit();

  // Success message verification
  const successMsg = page.locator('.x-paging-info');
  await expect(successMsg).toBeVisible();
  await expect(successMsg).toContainText('Lead created successfully');

});
```

```
test('should display validation error when required fields are missing', async () => {  
  // Intentionally omit Company Name and Phone Number  
  await createLeadPage.fillFirstName('Lakshmi');  
  await createLeadPage.fillLastName('Reddy');  
  await createLeadPage.selectCountry('India');  
  await createLeadPage.selectState('Andhra Pradesh');  
  await createLeadPage.fillCity('Visakhapatnam');  
  await createLeadPage.fillEmail('lakshmi.reddy@example.com');  
  
  await createLeadPage.submit();  
  
  // Expect error messages next to the missing required fields  
  const companyError = page.locator('#ext-gen602 ~ .errorMsg');  
  const phoneError = page.locator('#createLeadForm_primaryPhoneNumber ~ .errorMsg');  
  
  await expect(companyError).toBeVisible();  
  await expect(companyError).toContainText('Company Name is required');  
  
  await expect(phoneError).toBeVisible();  
  await expect(phoneError).toContainText('Primary Phone Number is required');  
});  
  
test('should reject an invalid phone number format', async () => {  
  await createLeadPage.fillCompanyName('Bangalore Innovations');  
  await createLeadPage.fillFirstName('Rohit');  
  await createLeadPage.fillLastName('Sharma');  
  await createLeadPage.selectCountry('India');  
  await createLeadPage.selectState('Karnataka');  
  await createLeadPage.fillCity('Bengaluru');
```

```
// Invalid phone (alphabets included)
await createLeadPage.fillPhoneNumber('ABCD1234');
await createLeadPage.fillEmail('rohit.sharma@bangalore.in');

await createLeadPage.submit();

const phoneError = page.locator('#createLeadForm_primaryPhoneNumber ~ .errorMsg');
await expect(phoneError).toBeVisible();
await expect(phoneError).toContainText('Please enter a valid phone number');
});

test('should allow lead creation with optional fields left blank', async () => {
  await createLeadPage.fillCompanyName('Kerala Agro Ltd');
  await createLeadPage.fillFirstName('Meera');
  await createLeadPage.fillLastName('Nair');
  await createLeadPage.selectCountry('India');
  await createLeadPage.selectState('Kerala');
  await createLeadPage.fillCity('Kochi');
  await createLeadPage.fillPhoneNumber('04841234567');
  await createLeadPage.fillEmail('meera.nair@keralaagro.in');

  // Skip optional fields like "Annual Revenue" and "Description"
  await createLeadPage.submit();

  const successMsg = page.locator('.x-paging-info');
  await expect(successMsg).toBeVisible();
  await expect(successMsg).toContainText('Lead created successfully');
});
```

```
test('should retain entered data after a validation failure', async () => {  
  await createLeadPage.fillCompanyName('Hyderabad Enterprises');  
  await createLeadPage.fillFirstName('Sanjay');  
  await createLeadPage.fillLastName('Patel');  
  await createLeadPage.selectCountry('India');  
  await createLeadPage.selectState('Telangana');  
  await createLeadPage.fillCity('Hyderabad');  
  // Omit phone number to trigger validation error  
  await createLeadPage.fillEmail('sanjay.patel@hydent.com');  
  
  await createLeadPage.submit();  
  
  const phoneError = page.locator('#createLeadForm_primaryPhoneNumber ~ .errorMsg');  
  await expect(phoneError).toBeVisible();  
  
  // Verify that previously entered fields still contain the same values  
  await expect(createLeadPage.companyNameInput).toHaveValue('Hyderabad Enterprises');  
  await expect(createLeadPage.firstNameInput).toHaveValue('Sanjay');  
  await expect(createLeadPage.lastNameInput).toHaveValue('Patel');  
  await expect(createLeadPage.emailInput).toHaveValue('sanjay.patel@hydent.com');  
});  
});
```